Assessment 1 — Retail Order Processing

Environment: PyCharm
Suggested duration: 3 Hours

Create a project:

```
python_assessment_1
```

Data File 1 — `customers.csv`

```
customer_id,customer_name,city,segment,active
C101,North Star Retail,Hyderabad,Corporate,True
C102,Blue Mart,Mumbai,Retail,True
C103,Daily Basket,Bangalore,Retail,False
C104,Prime Stores,Hyderabad,SMB,True
C105,Fresh Point,Delhi,Corporate,True
C106,Quick Buy,Pune,Retail,True
C107,Central Bazaar,Mumbai,SMB,False
```

Data File 2 — `orders.csv`

```
order_id,customer_id,order_date,product,category,quantity,unit_price,stat
1001,C101,2026-01-05,Laptop,Electronics,1,65000,Delivered
1002,C102,2026-01-12,Mouse,Accessories,3,1500,Delivered
1003,C103,2026-01-18,Monitor,Electronics,2,18000,Cancelled
1004,C101,2026-02-02,Keyboard,Accessories,4,3000,Delivered
1005,C104,2026-02-08,Office Chair,Furniture,2,9000,Pending
1006,C105,2026-02-14,Laptop,Electronics,1,72000,Delivered
1007,C102,2026-03-03,Headphones,Accessories,2,4500,Pending
1008,C103,2026-03-07,Monitor,Electronics,1,19000,Delivered
1009,C104,2026-03-15,Desk,Furniture,2,14000,Delivered
1010,C101,2026-03-21,Tablet,Electronics,2,28000,Delivered
1011,C105,2026-04-04,Keyboard,Accessories,5,3200,Delivered
1012,C106,2026-04-11,Office Chair,Furniture,1,9500,Pending
```

```
1013,C102,2026-04-17,Tablet,Electronics,1,30000,Delivered
1014,C104,2026-05-01,Mouse,Accessories,5,1600,Cancelled
```

Data File 3 — `products.json`

```
[
  {
    "product_id": 201,
    "product_name": "Laptop",
    "category": "Electronics",
    "price": 65000,
    "stock": 12,
    "supplier": {
      "name": "Tech Source",
      "city": "Hyderabad"
    },
    "colors": ["Black", "Silver"]
  },
  {
    "product_id": 202,
    "product_name": "Monitor",
    "category": "Electronics",
    "price": 18000,
    "stock": 8,
    "supplier": {
      "name": "Vision Traders",
      "city": "Mumbai"
    },
    "colors": ["Black"]
  },
  {
    "product_id": 203,
    "product_name": "Office Chair",
    "category": "Furniture",
    "price": 9000,
    "stock": 15,
    "supplier": {
      "name": "Comfort Works",
      "city": "Pune"
    },
    "colors": ["Black", "Grey"]
  },
```

```
  {
    "product_id": 204,
    "product_name": "Keyboard",
    "category": "Accessories",
    "price": 3000,
    "stock": 30,
    "supplier": {
      "name": "Tech Source",
      "city": "Hyderabad"
    },
    "colors": ["Black", "White"]
  }
]
```

Section A — Python Basics

1. Take product name, unit price and quantity from the user.
2. Calculate total amount.
3. If total amount is greater than or equal to `50000`, apply 10% discount; otherwise apply 5%.
4. Display original amount, discount and final amount using f-strings.
5. Ask for stock quantity and display:
   - `High Stock` for stock above 20
   - `Low Stock` for stock between 1 and 20
   - `Out of Stock` for 0
6. Use a `for` loop to display numbers 1 through 10.
7. Use a `while` loop to reduce stock from 5 to 0.
8. Skip number 5 using `continue`.
9. Stop a loop when the value reaches 8 using `break`.

Section B — Lists, Tuples, Sets and Dictionaries

Use:

```
sales = [65000, 4500, 36000, 12000, 18000, 72000, 9000]
```

10. Find total sales.
11. Find highest and lowest sale.
12. Display sales greater than `20000`.

13. Sort sales from highest to lowest.
14. Create a set from:

```
cities = [
    "Hyderabad",
    "Mumbai",
    "Delhi",
    "Hyderabad",
    "Pune",
    "Mumbai"
]
```

and display unique cities.

15. Create a tuple containing:

```
Product ID
Product Name
Category
Price
```

and unpack it into individual variables.

16. Create a dictionary representing a customer with:

- ID
- Name
- City
- Orders
- Active status

17. Update the customer's city.
18. Increase order count by 1.
19. Add an email field.
20. Loop through all keys and values.

Section C — Functions and Lambda

21. Create a function `calculate_total(price, quantity)` .
22. Create a function `calculate_discount(amount)` that returns the discount.
23. Create a function accepting a list of sales and returning total sales.

24. Create a function accepting a product dictionary and returning `"Available"` or `"Out of Stock"`.
25. Use a lambda function to calculate the square of a number.
26. Use `filter()` with lambda to select sales greater than `20000`.
27. Use `map()` to extract customer cities from a list of dictionaries.
28. Use `set()` to get unique cities.
29. Sort a list of product dictionaries by price using `key=`.

Section D — OOP

Create a class:

```
Product
```

with:

```
product_id
product_name
price
stock
```

30. Initialize the values using `__init__()`.
31. Add a method `display_product()`.
32. Add a method `calculate_value()` returning:

```
price × stock
```

33. Create two product objects.

Create a child class:

```
DiscountedProduct
```

34. Inherit from `Product`.
35. Add `discount_percent`.
36. Use `super()` to initialize parent values.
37. Override `display_product()` so the discounted product also displays its discount.

Create another class:

`PremiumProduct`

38. Give it its own version of `display_product()` .
39. Put `Product` , `DiscountedProduct` and `PremiumProduct` objects into a list and call `display_product()` for each to demonstrate polymorphism.

Section E — Text, CSV and JSON Files

40. Create `activity.txt` .
41. Write three processing messages to it.
42. Append one additional message.
43. Read and display the complete file.

Using `customers.csv` :

44. Read the CSV using the `csv` module.
45. Store the records in a Python list.
46. Display only active customers.
47. Display only Hyderabad customers.

Using `products.json` :

48. Read the JSON file.
49. Display all product names.
50. Display products costing above `10000` .
51. Display supplier name and supplier city for every product.
52. Extract all colors into one list.
53. Display unique colors.
54. Increase stock of product ID `202` .
55. Write the modified data back to a new file:

`products_updated.json`

Section F — User-Defined Module

Create:

`retail_utils.py`

56. Add functions for:

- calculating order total,
- calculating discount,
- checking stock.

Create:

`main.py`

57. Import the module.
58. Call all three functions.
59. Import one function directly and call it without the module name.
60. Import the module using an alias.

Section G — Pandas Data Processing

Using `orders.csv` :

61. Read the CSV into a Pandas DataFrame.
62. Display first 5 rows.
63. Display column names and datatypes.
64. Display shape of the DataFrame.
65. Filter only Delivered orders using `query()` .
66. Filter orders where quantity is greater than 2.
67. Calculate a new column:

```
order_value = quantity × unit_price
```

68. Count orders by status.
69. Convert `order_date` to datetime.
70. Create a `month` column.
71. Find order count by month and status.
72. Find total order value by category.
73. Find average order value by category.

74. Sort orders by order value descending.

Section H — Pandas Join and Aggregation

Read:

```
customers.csv
orders.csv
```

75. Perform an inner join using `customer_id`.
76. Display:

- order ID
- customer name
- city
- product
- status
- order value

77. Display only Delivered orders from Hyderabad.

78. Calculate by city:

- total orders
- total sales
- average order value

79. Sort city results by total sales descending.

80. Write the result to:

```
city_sales_summary.csv
```

81. Write the same result to:

```
city_sales_summary.json
```

using record-oriented JSON.

Section I — NumPy/Pandas Enrichment

Using the joined DataFrame:

82. Create a column `order_size` using NumPy:

```
order_value >= 50000 → High
order_value >= 15000 → Medium
otherwise            → Low
```

83. Create a new column:

```
priority_customer
```

where Corporate customers are marked `Priority`, others `Normal`.

84. Save the enriched dataset as:

```
enriched_orders.csv
```

Assessment 2 — Logistics & Employee Data Processing

Environment: PyCharm
Suggested duration: 3 Hours

Data File 1 — `shipments.csv`

```
shipment_id,hub_id,shipment_date,destination,weight,status,cost
S1001,H01,2026-06-01,Hyderabad,12.5,Delivered,850
S1002,H02,2026-06-03,Mumbai,8.0,In Transit,620
S1003,H01,2026-06-06,Bangalore,18.5,Delivered,1250
S1004,H03,2026-06-10,Delhi,5.5,Pending,480
S1005,H02,2026-07-02,Pune,22.0,Delivered,1600
S1006,H03,2026-07-06,Hyderabad,10.0,In Transit,740
```

```
S1007,H01,2026-07-12,Mumbai,7.2,Delivered,590
S1008,H02,2026-08-01,Delhi,15.5,Pending,1100
S1009,H03,2026-08-08,Bangalore,11.8,Delivered,920
S1010,H01,2026-08-14,Pune,25.0,Delivered,1750
S1011,H02,2026-09-02,Hyderabad,6.5,In Transit,520
S1012,H03,2026-09-09,Mumbai,14.0,Delivered,1050
```

Data File 2 — `hubs.csv`

```
hub_id,hub_name,city,manager
H01,South Logistics Hub,Hyderabad,Naveen
H02,Western Distribution Hub,Mumbai,Leena
H03,Central Freight Hub,Nagpur,Kabir
H04,Eastern Delivery Hub,Kolkata,Sonia
```

Data File 3 — `employees.json`

```
[
  {
    "employee_id": 301,
    "name": "Irfan",
    "department": "Operations",
    "salary": 68000,
    "skills": ["Python", "SQL"],
    "performance": {
      "rating": 4.3,
      "tasks_completed": 42
    }
  },
  {
    "employee_id": 302,
    "name": "Tanvi",
    "department": "Analytics",
    "salary": 82000,
    "skills": ["Python", "Pandas", "Power BI"],
    "performance": {
      "rating": 4.7,
      "tasks_completed": 55
    }
```

```
  },
  {
    "employee_id": 303,
    "name": "Yusuf",
    "department": "Operations",
    "salary": 72000,
    "skills": ["SQL", "Airflow"],
    "performance": {
      "rating": 4.1,
      "tasks_completed": 38
    }
  },
  {
    "employee_id": 304,
    "name": "Lavanya",
    "department": "Engineering",
    "salary": 95000,
    "skills": ["Python", "Spark", "SQL"],
    "performance": {
      "rating": 4.8,
      "tasks_completed": 61
    }
  }
]
```


Assessment Tasks

Python Fundamentals

1. Accept shipment weight and cost from the user.
2. Calculate cost per kg.
3. Classify:
    - weight above 20 → `Heavy`
    - 10–20 → `Medium`
    - below 10 → `Light`
4. Use loops to display shipment numbers from 1–20.
5. Display only even shipment numbers.
6. Use a while loop to simulate five processing attempts.

Collections

7. Create a list of shipment costs.
8. Calculate total, average, minimum and maximum.
9. Remove duplicate destination cities using a set.
10. Store one shipment using a dictionary.
11. Store multiple shipments using a list of dictionaries.
12. Filter list records by status.

Functions/Lambda

13. Create a function calculating shipping cost.
14. Create a function returning shipment category based on weight.
15. Use lambda/filter to select Delivered shipments.
16. Use map to obtain all destinations.
17. Sort shipment dictionaries by cost.

OOP

18. Create a `Shipment` class.
19. Add constructor values for shipment ID, destination, weight and cost.
20. Add a display method.
21. Create `ExpressShipment` inheriting from `Shipment`.
22. Override a method to add an express surcharge.
23. Demonstrate polymorphism using normal and express shipment objects.

JSON

24. Read `employees.json`.
25. Display all employee names.
26. Find employees earning above `75000`.
27. Display employees knowing Python.
28. Display employee name and performance rating.
29. Find employees whose rating exceeds `4.5`.
30. Sort employees by rating descending.
31. Add one skill to employee 303.
32. Save the modified JSON.

Modules

33. Create:

```
shipment_utils.py
```

containing reusable shipment functions.

34. Import them into `main.py`.
35. Use both normal import and specific-function import.

Pandas

36. Read `shipments.csv`.
37. Filter Delivered shipments.
38. Count records by status.
39. Convert shipment date to datetime.
40. Count shipments by month and status.
41. Calculate average cost by destination.
42. Calculate total weight by destination.
43. Read only selected columns using a dynamic `usecols` list.
44. Read `hubs.csv`.
45. Inner join shipments and hubs using `hub_id`.
46. Find total shipment cost handled by each hub.
47. Find total number of shipments handled by each hub.
48. Sort hubs by total shipment cost.
49. Save results to CSV.
50. Save results to JSON.

Pandas/NumPy Enrichment

51. Add:

```
cost_per_kg = cost / weight
```

52. Use NumPy to classify:

```
cost >= 1200 → Expensive
otherwise    → Normal
```

53. Add a shipment performance category based on both status and cost.

54. Save the final enriched dataset as:

shipment_analysis.csv